Meeting 18/03/2026

Summary

Action Items & Next Steps

- [ ] Students to develop a high-level design and architecture proposal for the bug triaging system ▸ ▸
- [ ] Students to create a POC to validate the proposed design before proceeding to detailed design stage ▸
- [ ] Students to develop a project timeline and plan for task distribution among team members ▸
- [ ] Students to explore and decide on UI/frontend framework and technologies ▸ ▸
- [ ] Students to research data storage fundamentals (storage hardware, RAID, servers, data center components) instead of databases ▸ ▸
- [ ] Students to evaluate single agent vs multi-agent system approach with pros and cons analysis ▸ ▸
- [ ] Students to explore agent communication mechanisms and orchestration strategies ▸
- [ ] Team to create WhatsApp group for ongoing communication ▸ ▸
- [ ] Team to set up recurring meetings once every three weeks ▸ ▸

Problem Statement Clarification

Core Challenge: Develop an agentic AI system to automate bug triaging across multiple disconnected bug tracking platforms (Jira, Bugzilla, Confluence, etc.) used by different teams within HPE's complex enterprise environment ▸ ▸

Real-World Context: HPE's hybrid cloud solutions are composite systems involving multiple components (servers, storage, networking, hypervisors, Kubernetes, cloud management platforms), each with separate teams and bug tracking systems. When customers log issues, the system must automatically

identify affected components, assign bugs to appropriate teams, and track resolution across all systems while presenting a unified view to customers ▸ ▸ ▸

Example Scenario: If a customer cannot create a VM in the private cloud platform, the failure could be caused by storage, server, or cloud management issues. The system should automatically assign the bug to all affected component teams, track each team's progress in their respective systems, and provide consolidated status updates ▸ ▸ ▸

Key Requirements:

- Fetch bug information from multiple heterogeneous data sources ▸
- Provide consolidated view across various bug tracking platforms ▸
- Track bugs to completion across multiple component teams ▸
- Support different personas (customers, executives, internal teams) with appropriate views ▸ ▸
- Gather all issues affecting a particular customer problem ▸
- Monitor SLA compliance and contract obligations ▸

Student Progress Review

Pain Points Identified:

- Manual triaging across disconnected systems ▸
- Lost tracking of bug states as teams use different platforms ▸
- Operational overhead from context switching between multiple repositories ▸
- Scattered data and manual mis-routing ▸

AI/ML Concepts Explored:

- Large Language Models (LLMs) and potential use of OpenAI, Claude, or open-source models ▸
- Tokenization and embeddings for text processing ▸
- Classification agents for severity assessment, team assignment, and timeline determination ▸
- CRUD agents and routing agents for automation ▸

- Tool calling and function calling for API interactions ▸

Database Technologies Explored (Note: This was a misunderstanding - team expected storage hardware fundamentals instead):

- Relational databases (PostgreSQL) for structured data and records ▸
- Graph databases (Neo4j) for tracking relationships and bug lineage across systems ▸
- In-memory cache (Redis) for fast querying and preventing duplicate bug processing ▸
- Message queues (Kafka) for handling concurrent bug submissions and agent failures ▸

Implementation Technologies:

- Python as main programming language for AI agents ▸
- Jupyter Lab or Google Colab for model testing and experimentation ▸
- Docker for running components in isolated containers ▸
- Kubernetes for scaling and managing multiple containers ▸
- Linux as base environment for system deployment ▸

Clarification on Data Storage Fundamentals

Misunderstanding Identified: Students researched databases when the team expected them to study storage hardware fundamentals ▸

Expected Topics:

- Block storage vs object storage ▸
- RAID configurations ▸
- Server and storage technologies ▸
- Data center components and architecture ▸
- Hybrid cloud infrastructure basics ▸

Architecture & Design Requirements

Reference Architecture Provided: HPE team shared a reference architecture as guidance, not a final design. Students must develop their own high-level architecture ▸ ▸ ▸

Key Components in Reference:

- UI frontend layer ▸
- Gateway for REST API access ▸
- Orchestrator module to coordinate multiple agents and connectors ▸
- Data plane for metadata and bug information storage ▸
- MCP servers for fetching information from external data sources ▸

Design Considerations:

- Apply system design patterns ▸
- Focus on scalability and performance ▸
- Consider how to map identified technologies to the problem statement ▸ ▸
- Create component diagrams showing building blocks and their relationships ▸

Multi-Agent System Considerations

Key Questions to Explore:

- Single agent vs multiple agents: evaluate pros and cons ▸ ▸
- Agent communication mechanisms and protocols ▸ ▸
- Agent monitoring and lifecycle management ▸
- Orchestrator or planning agent requirements ▸
- Level of human involvement in the workflow ▸

Determinism Requirements: System must provide deterministic responses (e.g., "bug state is open") rather than probabilistic ones (e.g., "could be open or closed") ▸

Guidance Provided: Multiple agents confirmed as necessary due to system complexity, but students should go through the evaluation process to understand why ▸ ▸ ▸

Frontend Considerations

Open Question: No progress shown on UI/frontend technology selection ▸

Required Exploration:

- Frontend framework selection ▸
- What to showcase in the prototype ▸
- How to present information for different personas ▸

Communication & Meeting Planning

- Team to create WhatsApp group for asynchronous communication and questions between meetings ▸ ▸
- Recurring meetings to be scheduled once every three weeks ▸ ▸
- Students can ask questions in WhatsApp group without waiting for meetings ▸

Project Timeline

Program runs until May, requiring students to plan work allocation and bandwidth accordingly ▸

Notes

Transcript

a clear picture of how bugs are actually classified, allocated, and dragged inside a large enterprise like HP. So before touching any technology we started here, we explored what bug triaging actually means in practice. So the main pain points like manual triaging across disconnected systems, losing track of all the bug states as each team uses a different bug platform, bug management platform like Jira or Bugzilla or any other repo.

and the operational overhead that forces engineers to context which constantly between these multiple repositories, multiple systems. So that gave us a why behind this architecture of what book triaging really is. So, So these are the problems like scattered data, manual mis-surfing, loss of operational overhead. And we started thinking through the solutions through it and the technology that will be subsequently used in these.

We also explored, touched upon the topics briefly, like you shared the topics like AIML, Python, virtualization. So these topics were quite broad for us. So we tried to map those topics. to this particular part problem statement and the architecture you share. So if someone could take on this AI toolbar.

Sure, sure. Thank you, Kulthum. So the main task of our project is to identify, track, and evaluate bugs using agent existence. So up until now, We have been doing it manually, but when we scale these projects to enterprise grade, The manual process becomes quite long. Like one person raises the issue, another one has to redo it, then it has to be assigned, and then it has to be worked on. So what we can achieve with agentic system now is we can automate this process through various agents like classification agent, a CRUD agent, a routing agent.

So for these agents to work the foundations of the IML that we touched upon, we explored what LLMs are basically and we explored how we could use them in our project. So we thought of using APIs of like OpenAI, Qlod or any open source models. We also studied upon tokenization and the process of embeddings, how these LLMs work, like they take all the text input, breaks it into chunks, then represents it as numbers because AIML is basically almost like multiplication.

Then it is passed through a classifier that assigns labels to it like what are we looking upon, like what is the severity of our bug, what is the production team that needs to handle it and what can be the time period about how long we can delay this or how quickly we need to resolve this. These were the basic topics that we touched upon the AIML concepts. Then comes the data storage fundamentals. So I would like to hand off it now.

is So, this slide is about the different ways the data is stored depending on our need. Like we have first mentioned the relational database like Postgre and that will be used when data is well organized in table and it needs to be accurate and consistent like for storing important rules and records we will be using this. And then we have the graph database. The graph database like Neo4j will be used when we will need to understand the between the data like how different items are linked we'll be having different connectors across so whenever we need to prepare something a kind of lineage kind of relationship when bugs come across different system so in that case we'll be requiring the graph database after that we have an in-memory cache database like redis that is for the fast querying like caching so whenever like multiple bugs are reported at the same time so in order

to prevent like so that the system does not have to same bug again and again in order to control the rate of the api calls we use the redis database and after this we have message queue and for that we will be using kafka that is the event was like a lot of bugs can come at the same time and in case any time as agent will not work so that kafka will be storing the queue of message queue and handling overall the main idea of the each type of storage is the data is used because one single system cannot handle every situation efficiently.

So each storage technology is basically based on what shape of data in excess patent. Next slide.

Sorry to interrupt. When we said the data storage fundamentals, we didn't mean database or cache and things like this. What we meant was the storage and the server. uh... domain So what is RAID or what are the server or the storage technologies. So that's what we meant. Storage hardware as such. things related to data storage, not really database. So probably I think there was a misunderstanding in that regard.

Like what is BlockStore? Thank you. object storage and things like that. So that's what we meant. What are the building basics of a data center? What are the components or the data storage technologies that we have inside the data center. So we went from that perspective, not really database. Probably you can be some red by it. It's time. Yeah. can go through like what are the what comprises of hybrid cloud architecture I think some some material was shared with you right so probably can take clues from that and Like what forms a data center and what are the hardware like storage and servers.

So around that you would wanted to go through those fundamentals. Okay sir.

So basically in this slide like what all technologies or what we are going to implement in this what all technique will be needed so firstly basic python like for our main programming language for both AI for making our agent and then we'll be using Jupyter lab or the google colab for testing our models or experimenting it before inting integrating into the main system also for deployment infrastructure will be using docker kubernetes or Linux docker for running each component separate and for the separate containers so that they are isolated and we can manage easily and then will be kubernetes so it will be helpful in scaling and managing multiple containers in a large systems and then which will be our base

environment where our system will be running and it will help us manage processing, networking, deployment, and efficiency.

So these are the things that we explored based on your email.

Like what we tried to do is to map the topics to the problem statement because the topics were quite broad to explore. AI, ML has the entire thing, machine learning, deep learning. So what we, based on our understanding of problem statement, like what generative AI, what agentic AI kind of would be used in the problem statement, we tried to map. through that, like tool calling, function calling, embedding, so that we could write better system prompts or understand-like, we didn't want to completely abstract the way that API called.

You want to understand how tokens will be used, how embeddings will be used. So that's what we tried to explore during these days. Similarly, to the data storage part, that we couldn't exactly map to what you expected. That's what we tried. Now further, we were expecting that if we could discuss if this part is we could discuss on the architectural level. so that we could dive deep into these components or we have to stick to the fundamentals still.

Yeah, I think this is good. This is a good start and I did not see anything on the UI front. Is there some thoughts about it you guys did or that's something you want to explore further?

Like to the front end, what exactly we will be showcasing in the prototype.

Which technologies that you will explore and which one will you be adopting? framework and things like that Yes, Shivansh, could you please show the architectural diagram?

So this was the architecture diagram that you showed. So primarily, you said that we will be discussing architecture in the next meeting once the fundamentals were cleared. So, once we have a clarity of this architecture, what exactly it is, what your understanding of the problem statement is and what are you expecting from us from this architecture? Can you do more?

Yeah. So the problem statement that we have shared with you, is it clear to you? what needs to be done. Uh...

Like from our understanding, we have understood what pathrahajying is, what are the pain points that we discussed in the first slide. But still we would like to hear from your point of view. So that we could map exactly to...

Yeah, it's the same like what we had explained before. The problem statement is that there will be multiple data sources from where bug tracking systems will be available. So you have to develop a framework which should be able to affect the issues from across those bug data sources basically. and to be able to have access to even for a particular case let's say there is a certain bug and what are the list of customer cases around that particular bug and within various components if there are tracking systems different for each component in an application right so there will be various demons of services and maybe first there will be different teams for each like for example in a data storage project the team could be altogether different, located in a different place and using a different bug tracking system.

So this UR system should be capable of grabbing the information from all these multiple data sources for tracking the bug information and should be able to show a consolidated view across these various data sources for getting clarity on the particular bug information. and should have a detailed information about the entire bug, regardless of the various data source platforms that we talked about like Jira or Bugzilla.

Confluence, whatever.

Kind of like a unified platform for all the management platforms. And what about agents in these? Yeah, I mean using agents, how do we solve this problem, right? So I think yeah, I see. some of the agents there.

Maybe you can proceed with explaining the architecture diagram. Maybe it will be more clear. Actually, this diagram is something which you have created or this is the reference architecture that we provided you.

The reference architecture.

This is the architecture we provided, right?

Right, right.

Yeah, so this is just a reference for you. So ultimately you own the design and you have to come up with a high-level design. This is just a clue for you in terms of how it can be. So there will be a UI front-end and there will be a gateway so that the REST APIs can be accessed from endpoint to an endpoint for a user or a UI to access it. And there will be an orchestrated module which will be able to orchestrate across these multiple agents, connectors.

all those bug repositories that it has to talk to. So that will take care of the overall control plane part. And then there is the data plane where you will store your metadata and all of this bug information, summary data, and all of that. So then there will be a certain MCP servers that you may have to implement in order to fetch information from external data sources. So all of those, this was just a reference architecture just to guide you.

So use this as a clue and you have to come up with your own architecture component diagrams what what all things may be possibly there in the world high level architecture and I think you guys should have to come up with a architecture proposal of your solution. When you envision the problem, you kind of in your earlier slides view, you went through like what are the possible constructs that you guys will be using for each of the building blocks.

How you guys want to put it together and showcase as a higher level architecture, I think that should be your next task.

Yeah, come up with a high level design and also target for coming up with a sort of a small POC in order to validate your design and then you can proceed to the detailed design stage after that. So that could be sort of a roadmap for you. and also see how you can plan the timelines based on the like this program till may i think it will be so based on the timelines and how much bandwidth you will be able to spare on it because accordingly please come up with a plan and expecting that you guys will distribute the tasks among yourselvesMother goes to climbing, Chad.

I... So I wasn't there the last time you guys met. This is Barad.

I sent a little bit of Either maybe you guys haven't understood the ask or it's not an ask. It's essentially the problem that we are sort of trying to solve here.

So if I may take a couple of minutes to sort of explain, give a little background and hopefully that should give you some idea as to why we need a triaging system, right?

So in the slides that you share, let's take one component. Let's take Redis as a triage. as a product.

So let's assume that we're all part of the company that is developing Redis, the database, in-memory database. So we've given Dreadist to our customers and they're using it. And one of the customers finds an issue.

In a simplistic world, the customer logs a defect against Redis saying, hey, something is not working or I'm not getting the performance that the performance is not good, right?

Performance is not good. out of memory issues or whatever, like one of those issues they find and they log a bug. Because it's a simplistic world, one of us is the developer. We pick it up, we go analyze the bug, we look at our code, We produce it in our lab or in our environment. We identify a fix and once it's fixed, we release a patch or the fix goes as part of our next minor or a major release.

Customer then gets access to that new patch in release. they upgrade their system and their issue is resolved. So this is the life cycle of product that is developed, picked up by a customer or a user. They're using it, they find an issue, how it gets fixed, and they get a new listing. But then enterprise systems are much more complex. They are not made up of simplistic, single component deployments.

They involve multiple sort of components. And all of us here, at least on the HPE side, we're all part of the hybrid cloud organization, Some of the products that we have are complex and composite in nature, meaning that it's not just one component like Redis. We have HP being an infrastructure vendor. We sell servers, networking equipment, enterprise storage systems. And from a platform standpoint, we have Hypervisor that we offer.

We have control plane components that are made up of other software components like operating systems. We also have Kubernetes clusters, containerized applications that are deployed in those Kubernetes systems, cloud management platforms. So many things, a composite complex system, a solution like a private cloud offering is made up of so many components. But from a customer standpoint, they just see a single interface, single solution, or a single product they are consuming.

So for them, if they are unable to create a virtual machine in the private cloud management platform, they just log an issue saying, I could not create the VM. But the thing is internally that failure as to why the virtual machine could not be created could be due to any of these components that I mentioned. Each of those components have separate owners. separate responsible people. It's a separate product line itself.

And they have separate systems that they use to manage the lifecycle of those components or products. Which is why you know the different systems that we talked about. Some teams use Jira, some teams use Eclipse, some teams use something else, and so on. But from a customer standpoint, they're just dealing with HPE. They're dealing with a single company. Now, obviously, we don't want to let the customer know that, oh, we forwarded this to the storage team.

this to the server team, they forwarded this bug to the Cloud Management Platform team, and they're working on it. That's not a good experience we want to be able to give. As soon as the customer logs a bug, we want an agent tick. workflow, agent API workflow to run that picks up the bug and then it tries to determine who is the component owner for that or which component is affected who is the owner for that component and automatically assigns that bug.

It could be more than one component, right? Creation of a VM failed because there was an issue with the server. There was also an issue with the storage. It could not allocate the necessary volume. So the bug is assigned against both of those. Now, we want to be able to track the issue to completion. And then we want to show that status to the customer that there are multiple components affected or causing the issue.

Both of them are being worked upon. And once both of them are fixed, we know that the next iteration or the patch that we'll be able to provide to fix the deployment, fix whatever is deployed at the customer's location so that they can now get back to creating the VM that they wanted to. So the idea is simple. able to find out more the component owners, map the bug to the component using AgentiKI to run this workflow that does the job.

That's number one. We also want to be able to gather All the issues that is affecting a particular customer issue. This is on the other side. One is the customer logging the issue. The other one is how do we track multiple issues causing an outage? We want to be able to say, "Okay, all of these issues now are fixed. Now outage or these customer issues should go away in a particular patch or whatever." We want to be able to paint that picture also.

Both of these have frontend. Both of these use cases have backend agents and other components with MCP servers in question. to be able to get that information and paint the picture. So, yeah, I know I took over two minutes, but I wanted to sort of imagine that visually, right? This is a complex composite system. And then

because there are so many involved, we want to be able to effectively depict what is happening to an issue that is being faced by the customer and how perhaps it is being handled internally.

Now, the customer need not get full details of it, but then there are different personas, right? So customer persona is one. to see what are all the customer issues that we are, as a company, as a group, we are working on and at what stage are these issues at? Are we honoring the SLAs? Are we honoring the contracts that we have with our customers in terms of when the bug should be fixed and all of that?

So this agenting bug triaging system and bug resolution tracking system should be able to sort of provide this data to different personas. who are looking for information about this. So again, I'm not sure if this is what, it's a sort of the line that my colleagues took, which again, I'm not, I just wanted to sort of clarify that we are all on the same page as to what is expected here. And with that, I'm happy to answer any questions, but if you already knew this, Apologies for wasting your time the last four minutes sort of trying to explain what it is.

But yeah, I can answer any other questions. And guys, Rajesh, Madhukar, Divakar, please add if if there's more to this that I might have missed. Oh. Thanks for explaining the problem statement. Now we have a good understanding of what All of the pain points from a bug getting reported to how it's getting being resolved end to end and what are the POVs we need to solve like a customer POV like an executive POVs everything needs to be sorted Yes, we will be needing time to implement architecture front-end, back-end, and ATMs.

So could you please guide us on how should we approach the architecture? It would really help a lot.

This reference architecture should be a guideline for you. So utilize this and look at the design patterns that you want to apply and go through your system design basics and how it can be scaled. So keep scalability and performance also in mind and accordingly you need to go through and then come up with a high level design. Oh, focus. Any other questions last three minutes? Sir, I am the mentor of, university mentor for this team and his monomys.

So my question is, I am audible? Yeah.

I think the agent part of this project is more scriptural.

There is need to develop single agent or like it would be multi agents would be like some work classification or something else for developer rating or priority scoring. Yeah.

So it's a reference guideline here.

Here in the reference architecture, we have shown multiple agents here.

Each agent is being responsible for different aspect or different dedicated tasks that it is supposed to accomplish. So use this as a reference and see what works out best in the interest of performance and scalability for the overall solution to work. So we would like to keep it open-ended and for you to explore and with findings based on which you have decided on a certain approach. So that's the like You know, a guideline that we would like you to...

take back If I may add there, it will certainly not be single agent system. It will be multiple multi-agent system because there are multiple components out there and things like that. So explore what agentic systems that we can use and How... the agents communicate with each other and what may be required to monitor these agents, etc. Those are the angles you will have to think through when you design an agent-DKI system.

Yes, it is. I think the, so it's easy for us to answer your question, Anish. I believe that you're the one who asked the question, right? But I think what we are sort of expecting is you guys to go through the, Exercise off. You know, Does a single agentic system make sense? for a complex system such as this, or do you think having multiple agents makes sense. What are the pros and cons of both? That's number one.

If it's a single agent system, communication within the agent between different stages of the agentic AI workflows, how does that flow? If you carve out multiple agents, how do, as Rajesh pointed out, how do each of those agents communicate with each other? How do they get access to the tools in question? Who invokes the first agent? Will you have like an orchestrator or a planning agent? Or is it dependent on a user input?

How much of human involvement are you bringing in? and how do you make the decisions taken by the agent? deterministic and not probabilistic. When I say deterministic, If someone asks, what is the state of the bug? The agent with the

help of a model and the use of a tool should say this bug's current state is open. and not it could be open, it could be closed, it could be being created and all that.

You need to build determinism into the system. Can you achieve that with a single agent? Will multiple agents help you achieve higher levels of determinism in the system? Like I said, we can give you the answer, but I think we would like you to go through the sort of journey of... evaluating both these approaches and come up with a pros and cons sort of a thing as to why is the agent thing is good or a multi-agent thing is good.

What are the challenges? How do you sort of manage all of those agents, the lifecycle of those agents and all of that. So, yeah, just I think the journey is important is what I'm trying to say. Yes. I think we'll have to jump to the other. Oh yeah, we'll have to jump to the other meeting. Yeah, quickly to wrap up. So you guys explore whatever we just mentioned and we will create a WhatsApp group. You can share your numbers.

I think we all let other organizers might already have will, you know, take from them and then we'll create a WhatsApp group and then, you know, you guys can, you know, ask for any questions there. We don't have to wait until the next meeting. So we'll set up a recurring meeting as well, maybe once in two or three weeks if that is okay with the vendors. So yeah, we'll decide that kind of thing.

Yeah. Go ahead. Once in three weeks should be fine actually.

All right. Yeah. Thanks.

Thanks a lot, team. Time to get that. Thank you sir. -